
02 Laboratory Exercise - Matrix Operations (2D Arrays & Nested Loops)

Student Name: Edgardo B. Del Rosario Jr.

Year and Section: BSIT-1C

Course Learning Objectives:

C01: Comprehend and clarify the characteristics, functions, and applications of essential data structures (such as arrays, linked lists, stacks, queues, trees, and graphs).

C02: Create and execute data structures and algorithms in a programming language.

C04: Utilize suitable data structures and algorithms to address computational challenges.

Activity Tasks:

- Understand row-major order and multidimensional array traversal by manipulating a mathematical matrix.

Submission Contents/ Requirements:

1. You must include the actual source code file and/or screenshots of the running program showing its successful operation.
2. Please ensure that your written answers or entries are provided on new, separate sheets.

Your PDF must be named as:

Class_LastName_FirstName_LE02.pdf (Ex: IT1a_DelaCruz_Juana_LE02.pdf)

Failure to follow instructions (wrong format, missing screenshot, no code inside PDF) will result in score deduction.

Scenario

Image processing and game grids often rely on 2D arrays. Your task is to implement a basic matrix manipulation tool.

Requirements

1. Initialize a 3x3 integer matrix with the values: 1 2 3
4 5 6
7 8 9
- Implement a function `printMatrix(int arr[3][3])` to display the grid.
2. Implement a function `transposeMatrix(int arr[3][3])`.
- ☐ Transposition means swapping rows and columns.

☐ Example: Element at `[0][1]` (2) swaps with `[1][0]` (4).

☐ Diagonal elements (`[0][0]`, `[1][1]`, etc.) remain unchanged.
3. Display the matrix before and after transposition.

Sample Output

Original Matrix:

1 2 3
4 5 6
7 8 9

Transposed Matrix:

1 4 7
2 5 8
3 6 9

Criteria	Description	Points
Matrix Setup	Correct initialization and display of the 2D array in a grid format (Row-Major).	15 pts
Transposition Logic	Nested loops correctly swap <code>arr[i][j]</code> with <code>arr[j][i]</code> without overwriting data incorrectly.	25 pts
Code Efficiency	Usage of appropriate loop bounds to avoid out-of-bounds access.	10 pts
Total		50 pts

SOURCE CODE(TEXT):

```
/*
Developed by: DevEJ_Del_Rosario
Fullname: Edgardo B. Del Rosario Jr.
Class: BSIT-1C
=====
Project: 02 Matrix Operations (2D Arrays & Nested Loops)
Developed by: DevEJ_Del_Rosario
Copyright (c) 2026. All Rights Reserved.
=====
This code is the original work of DevEJ_Del_Rosario

Users may utilize this code as a guide or for educational purposes.
However, claiming this code as one's own or using it for commercial purposes
without proper citation or acknowledgment of the original creator is strictly prohibited.
=====

*/
//Call the needed Libraries
#include <iostream>
#include <chrono>
#include <thread>

//initiate Milliseconds for the delay duration
int ms =640 ;

//Adds Delay
void delay(int millisecs){
    std::this_thread::sleep_for(std::chrono::milliseconds(millisecs));
}

//For Printing The Original Matrix
void printMatrix(int arr[3][3]){
    std::cout<<"Original Matrix"<<std::endl;;
    for(int i=0; i<3; i++){
        for(int k=0; k<3; k++){
            std::cout<<arr[i][k]<<" ";
            delay(ms);
        }
        std::cout<<"\n";
    }
}
```

```
//For Transposing and Printing Matrix
```

```
void transposeMatrix(int arr[3][3]){  
    int temporarySpaceHolder;  
    int &x = temporarySpaceHolder;
```

```
  
    for(int i=0; i<2; i++){  
        for(int k=2; k>i; k--){  
            x = arr[i][k];  
            arr[i][k] = arr[k][i];  
            arr[k][i] = x;  
        }  
    }  
    x =0;  
    std::cout<<"\n\n\nNew Matrix"<<std::endl;;  
    for(int i=0; i<3; i++){  
        for(int k=0; k<3; k++){  
            std::cout<<arr[i][k]<<" ";  
            delay(ms);  
        }    std::cout<< "\n";  
    }  
}
```

```
int main() {  
    //Initiates Array Values  
    int n=1;
```

```
  
    //Initialize 2D Array as myArr[][]  
    int myArr[3][3]={};
```

```
  
    //Assigns values for myArr's coordinates using nested for-loop  
    for(int i=0;i<3;i++){  
        for(int k=0;k<3;k++){  
            myArr[i][k] = n;  
            n++;  
        }  
    }  
    //Run printMatrix()  
    printMatrix( myArr );
```

```
  
    //Run transposeMatrix()  
    transposeMatrix(myArr);
```

```
  
    return 0;  
}
```

PROGRAM SAMPLE OUTPUT:

```
✓ TERMINAL C/C++ Compile Run + - [ ] [ ] [ ] ...
PS C:\PROGRAMMING_AND_DEVELOPMENT\CODES> cd 'c:\PROGRAMMING_AND_DEVELOPMENT\CODES\Projects\Programming Languages\C++\Basic\School Projects\Laboratory Exercises\DSA\LE02\output'
PS C:\PROGRAMMING_AND_DEVELOPMENT\CODES\Projects\Programming Languages\C++\Basic\School Projects\Laboratory Exercises\DSA\LE02\output> & .\'.cpp.exe'

Original Matrix
1 2 3
4 5 6
7 8 9

New Matrix
1 4 7
2 5 8
3 6 9
PS C:\PROGRAMMING_AND_DEVELOPMENT\CODES\Projects\Programming Languages\C++\Basic\School Projects\Laboratory Exercises\DSA\LE02\output>

✓ OUTPUT
[2/7/2026, 11:37:33 PM] For C++ source files, IntelliSenseMode was changed from "windows-clang-x64" to "windows-gcc-x64" based on compiler args and querying compilerPath: "C:/msys64/ucrt64/bin/g++.exe"
[2/7/2026, 11:37:39 PM] For C source files, IntelliSenseMode was changed from "windows-clang-x64" to "windows-gcc-x64" based on compiler args and querying compilerPath: "C:/msys64/ucrt64/bin/g++.exe"

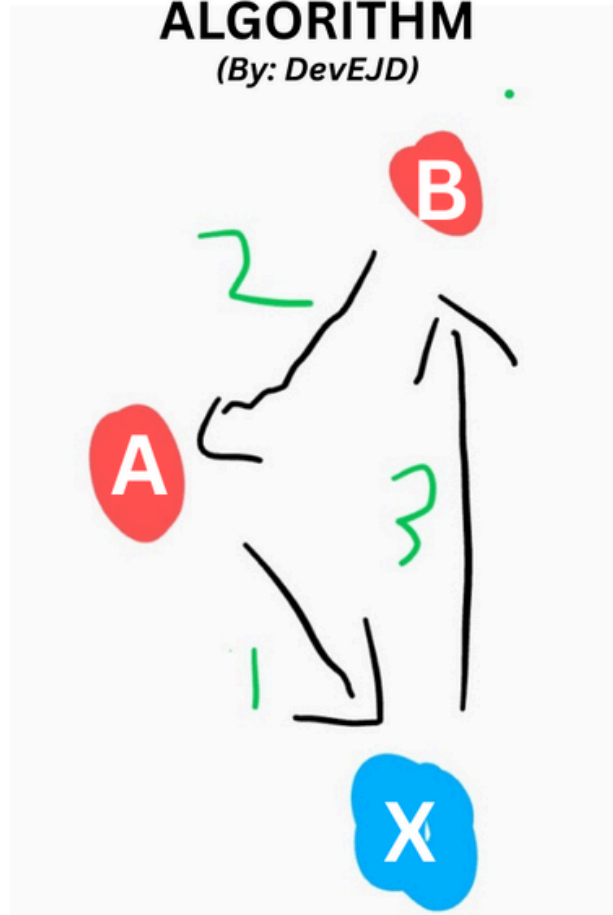
Spaces: 4 UTF-8 CRLF { } C++ Signed out Go Live Quokka windows-clang-x64
```

THE ALGORITHM DIAGRAM I FOUND OUT THAT HELPED ME
SOLVE
THE MATRIX TRANSPOSITION LOGIC:

VALUE SWAPING

ALGORITHM

(By: DevEJD)



LEGENDS:

- A = `myArr[i][k]`
- B = `myArr[k][i]`
- X = `temporarySpaceHolder`